

NAME:TANKISO MASOEBE

PROGRAMMING WITH C++2 WK7

1.Scenario Analysis: IT System Template – Inventory Management System

An Inventory Management System is an IT system designed to help an organization monitor, control, and manage its stock efficiently. A basic template for this system includes several important sections. First, the **Login and User Management** section allows authorized employees to access the system securely. Second, the **Product Management** section stores information such as product names, codes, quantities, prices, and suppliers. Third, the **Stock Management** section records stock received, stock sold, and current stock levels. Fourth, the **Supplier Management** section keeps supplier contact details and purchase information. The system should also include a **Reporting** section that generates reports on available stock, sales, low-stock items, and inventory movements. Finally, an **Alert System** can notify users when stock reaches a minimum level. This template helps organizations reduce errors, prevent stock shortages, improve record keeping, and make better decisions using accurate and up-to-date inventory information.

2.CONCEPT RESEARCH:

Template Specialization

Template specialization is a feature in C++ that allows programmers to create a specific version of a function or class template for a particular data type or condition. Normally, templates provide general code that works with different data types. However, some data types may require different behavior. Template specialization solves this problem by allowing programmers to define customized code for those specific cases. It improves flexibility and allows developers to handle special requirements efficiently. There are two main types: **full specialization**, which completely defines a template for a specific type, and **partial specialization**, which specializes a template for a group of related types. Template specialization is useful for creating reusable, efficient, and adaptable C++ programs.

3.TOOL PRACTICE:

Class Template in C++

A class template allows programmers to create a general class that can work with different data types. I created a simple class template in Visual Studio Code that stores and displays a value.

```
#include <iostream>
using namespace std;
```

```

template <class T>
class Box {
private:
    T value;

public:
    Box(T v) {
        value = v;
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Box<int> number(10);
    Box<string> text("Hello");

    number.display();
    text.display();

    return 0;
}

```

Using a class template helped me understand how C++ supports code reusability. The same class can handle integers, strings, and other data types without rewriting the code. This makes programs more flexible, organized, and easier to maintain.

4.DIAGRAM: Template Function for Data Processing

A template function allows the same function to process different types of data without writing separate functions for each data type. The following C++ template function can be used to display data:

```

#include <iostream>
using namespace std;

template <typename T>
void processData(T data) {
    cout << "Processed data: " << data << endl;
}

int main() {
    processData(100);
    processData(25.5);
    processData("Student Record");

    return 0;
}

```

The purpose of this template function is to make data processing flexible and reusable. It can accept integers, decimal values, strings, and other compatible data types. Using templates reduces code duplication, saves development time, and makes programs easier to maintain. This approach is useful when processing different types of information using the same operation.



Template Function for Data Processing

A simple C++ template function that can work with different data types.

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  // Template function for data processing
5  template <typename T>
6  void processData(T data) {
7      cout << "Processed data: " << data << endl;
8  }
9
10 int main() {
11     // Call the template function with different data types
12     processData(100);           // integer
13     processData(25.5);         // double
14     processData("Student Record"); // string
15
16     return 0;
17 }
```

What the Code Does

The template function `processData()` accepts any data type (using `template <typename T>`) and prints it to the console. In `main()`, the function is called with an integer, a double, and a string.

Sample Output

```
Processed data: 100
Processed data: 25.5
Processed data: Student Record
```

Purpose and Benefits



The purpose of this template function is to make data processing flexible and reusable. It can accept different data types without writing separate functions for each type. This reduces code duplication, saves development time, and makes programs easier to maintain. It is useful when working with different types of information using the same operation.